INDOOR AUTONOMOUS DELIVERY ROBOT · REPORT 2 OF 6

Object Detection with ArUco Markers

Marker design, OpenCV detection and pose estimation, accuracy, and use in mapping and delivery

Hüseyin Tozluyurt · ROS 2 Jazzy / Gazebo Harmonic simulation stack · November 2026

1. Role of ArUco markers in the robot

The robot recognises places with square fiducial markers from the OpenCV **ArUco DICT_4X4_50** dictionary (4×4 bits, 50 IDs). Each marker has a known ID, size and purpose, so a detection gives three things at once: *which* landmark is in view (no data-association problem), *where* it is relative to the robot (range and bearing from the camera pose), and *which way* it faces (used to place the delivery point in front of it). The markers are used by every part of the stack: the EKF-SLAM mapper adds them as landmarks, the delivery navigator uses them as beacons for localisation, and at every stop the robot turns to the marker and checks that the camera sees the correct ID before it reports *Delivered*.

32	1523	25.0 m	≈ 5–8 cm
markers in the two_buildings world	detections logged while mapping campus_full_map7	longest detection (1.0 m marker)	range error at 3 m, 0.30 m marker (synthetic test)

2. Markers and camera

Kind	Size (black square)	Count	Where
delivery	0.30 m	22	Behind every delivery pad / on tables (room A, room B, corridor rooms), 0.4–0.6 m high
room	1.00 m and 0.60 m	2 + 2	Far wall of each building (seen on entry) and inside the doorways
corridor	0.60 m and 0.40 m	4 + 2	Corridor walls, so the robot sees a landmark in long corridors

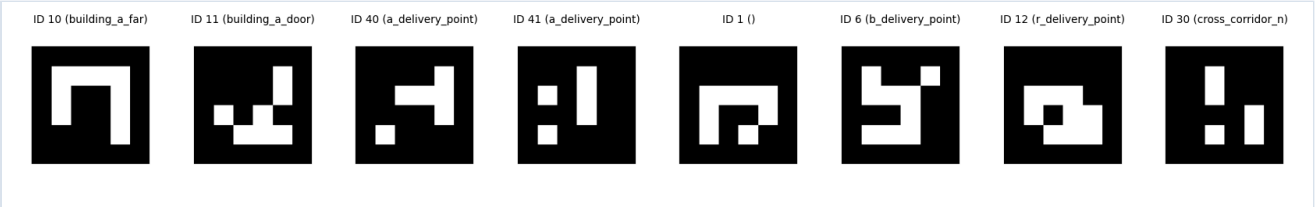


Figure 1. Examples of the DICT_4X4_50 markers used in the world (generated with the project's `aruco_utils.generate_marker`). Printable PNGs for the real robot are in `models/aruco_marker_<id>/meshes/`.

The markers are seen by a dedicated **mast camera** (`aruco_camera`): RGB 640×480 at 15 Hz with a horizontal field of view of 1.2 rad (69°), mounted above the cargo box at base_link +0.15 m in x and +0.55 m in z, looking forward. Topics: `/robot/aruco_camera/image_raw` and `/camera_info`. Marker poses come from `config/aruco_markers.yaml` (campus_area) and `config/aruco_markers_two_buildings.yaml`, both generated together with the Gazebo models by the scripts in `scripts/`.

3. Detection pipeline

1. **Detect** – the image is converted to grey and passed to `cv2.aruco` (the `ArucoDetector` class on OpenCV ≥ 4.7, `detectMarkers` on the Ubuntu 24.04 OpenCV 4.6; `aruco_utils.MarkerDetector` hides the difference). Result: four corner points and the ID of each marker.
2. **Filter** – markers whose mean edge is smaller than `min_pixels = 12` are dropped, and IDs that are not in the marker config are ignored (`known_markers_only`), which removes misreads.
3. **Pose** – `cv2.solvePnP` with `SOLVEPNP_IPPE_SQUARE` and the marker's real size gives the rotation `rvec` and translation `tvec` in the camera optical frame.
4. **Robot frame** – the translation is moved into base_link (`bx = 0.15 + t_z`, `by = −t_x`), giving **range** = √(bx²+by²) and **bearing** = atan2(by, bx). The marker normal (third column of the rotation matrix) gives the **facing** angle. Detections farther than 25 × marker size, or whose facing is more than 80° away from "towards the robot" (a pose flip of the planar PnP), are rejected.
5. **Noise model** – each observation carries σ_r = max(0.03, 0.8·range/pixels) and σ_b = 0.012 + 0.05/pixels rad, so far, small markers count less in the EKF. While the robot turns, the bearing noise grows with the turn rate (camera timing error), and above 0.5 rad/s marker bearings are not used.

The standalone ROS node `aruco_detector` publishes an annotated image, a `PoseArray`, a JSON list of detections and the current room; the mapper and navigator run the same code internally (`ArucoObserver`) so they do not depend on that node.

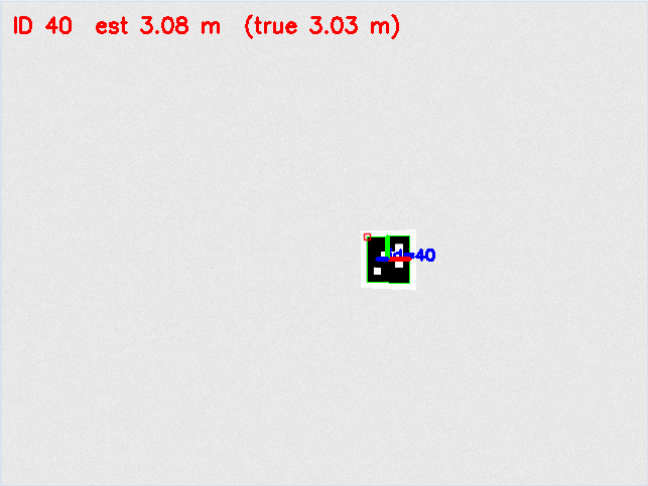


Figure 2. Synthetic test frame: ID 40 rendered through the mast-camera model at 3 m and turned 30°, detected and posed by OpenCV (axes drawn with `drawFrameAxes`).

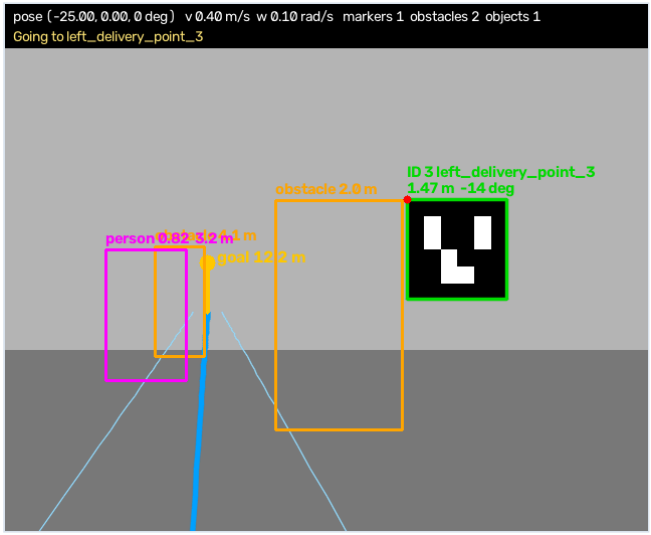


Figure 3. Robot vision overlay (`robot_vision`) in Gazebo: marker ID, name, range and bearing on the live camera, together with other detected objects.

4. Accuracy test with the real camera model

To measure the range error without the influence of SLAM, a 0.30 m marker was rendered with a perspective warp through the exact mast-camera intrinsics (f = 320/tan(0.6) = 468 px, 640×480), with light image noise, at distances from 1 to 9 m and turned 0°, 30° and 50° away from the camera. Each frame went through the same detection and `solvePnP(IPPE_SQUARE)` code. (Synthetic frames, not Gazebo renders; script: `reports/src/scripts/figs.py`.)

Distance	0° (face-on)	30°	50°
1 m	1.0 cm / 140 px	0.4 cm / 145 px	0.9 cm / 138 px
2 m	3.3 cm / 69 px	2.3 cm / 68 px	2.6 cm / 62 px
3 m	7.6 cm / 46 px	5.4 cm / 44 px	5.6 cm / 40 px
4 m	12.8 cm / 34 px	13.0 cm / 33 px	7.2 cm / 29 px
5 m	19.8 cm / 27 px	19.9 cm / 26 px	29.8 cm / 23 px
6 m	37.9 cm / 22 px	38.0 cm / 21 px	23.7 cm / 19 px
8 m	26.5 cm / 16 px	77.2 cm / 16 px	77.2 cm / 14 px
9 m	36.4 cm / 14 px	–	66.7 cm / 12 px

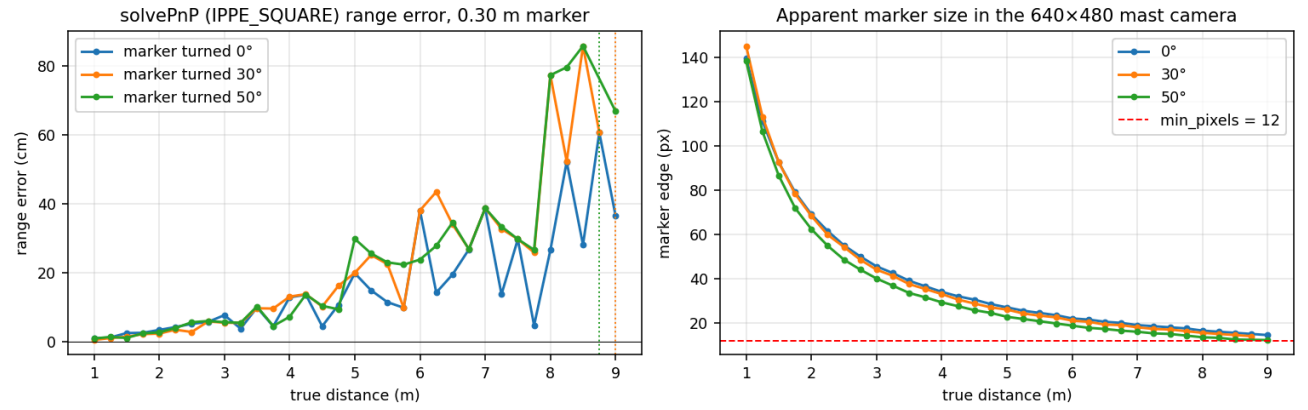


Figure 4. Left: range error of the PnP estimate grows roughly with the square of the distance and is always positive – most likely because the detected corners sit slightly inside the black border, so the marker looks smaller and farther away. Right: apparent marker edge in pixels; the 12 px limit is reached near 9 m for a 0.30 m marker.

Two results matter for the rest of the system. First, a 0.30 m delivery marker is accurate to a few centimetres within 3 m, which is exactly where the robot stands when it confirms a delivery (2.3 m stand-off). Second, beyond ~5 m the error reaches decimetres; this is why the EKF noise is scaled by range/pixels, and why large 0.6 m and 1.0 m markers are placed where the robot must localise from far away (room far walls, corridors).

5. Detections during a real mapping run

The mapper logs every accepted detection to `observations.csv`. For the final map `campus_full_map7` (two_buildings world) there are **1523** detections of **19** different markers. The median range is 5.5 m and the longest 25.0 m (a 1.0 m room marker).

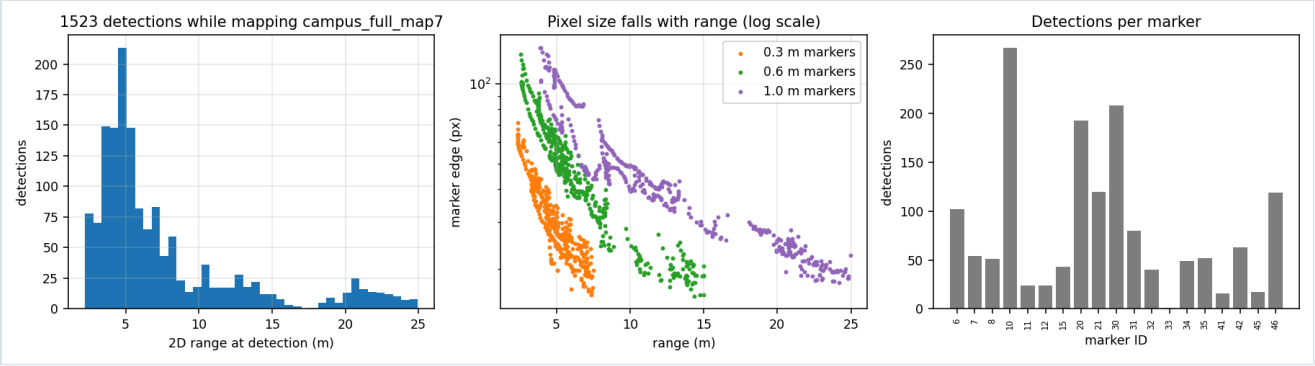


Figure 5. campus_full_map7 detections. Left: range distribution. Middle: pixel size against range for the three marker sizes – the big markers are what make 10–25 m detections possible. Right: detections per marker; room/corridor markers are seen most because they are visible from many places.

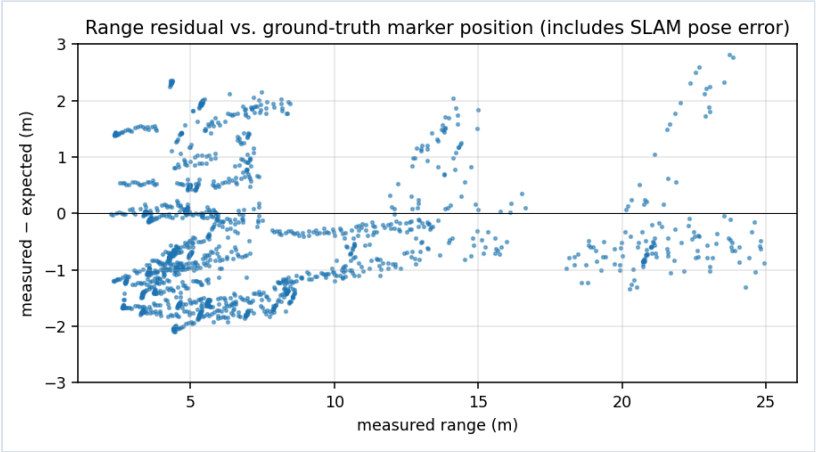


Figure 6. Measured range minus the distance from the SLAM pose to the true marker position (world file). The spread (median |residual| 0.98 m, 90th percentile 1.78 m) is dominated by the robot-pose and map-frame error of the SLAM run, not by the detector – compare with the centimetre-level error of the isolated test above.

6. Use at the delivery stop

When the navigator reaches a delivery point it turns to the stored facing of that point's marker and looks for the expected ID (or any marker of that table) for up to 2 s. The log line then says *marker N confirmed at d m*, or carries a *WARNING marker N not in view*; after the dwell time the order is reported *Delivered*. If the point cannot be reached, the navigator recovers its pose from markers and lidar, tries once more, and otherwise reports *Failed*. A stricter mode that refuses to deliver without the marker would be a one-line change. In the offline replica the robot stopped 0.08–0.16 m from the real floor pads with the right marker confirmed at every stop.

7. Moving to the real robot

- Print the markers from `models/aruco_marker_<id>/meshes/aruco_<id>.png` and measure the black square; put the measured size into the marker YAML – range scales directly with it.
- Calibrate the real camera (`camera_info`), set `camera_offset` to its true mount position, and keep the 0.30 m markers at the same height as in the simulation.
- Lighting and motion blur are the main real-world risks; the `dev/aruco_playground.py` tool lets you tune `DetectorParameters` (adaptive threshold windows, corner refinement) on recorded frames.

Summary. ID-based fiducials remove the hardest SLAM problem (data association) and give a reliable "am I at the right door?" check. The measured accuracy – a few cm at the delivery stand-off, decimetres at 5–8 m – matches the noise model the EKF uses.